

FINAL YEAR PROJECT · PHASE 1 · FINAL PRESENTATION · REVIEWS 1–3

EYEQ: A Hybrid Quantum–Classical Eye-Gaze Typing System on Commodity Webcams

PRESENTED BY

Devash S (23619110)

Batch – 4

Department of Computer Science and Engineering Specialised in Artificial Intelligence and Machine Learning

PROJECT GUIDE

Dr. Anandan R

Professor

Department of Computer Science and Engineering Specialised in Artificial Intelligence and Machine Learning

Introduction

- Text entry still assumes working hands and a physical keyboard — excluding users with motor impairments, and failing in hands-occupied or sterile settings such as operating theatres and laboratories.
- Research-grade eye tracking already solves the sensing problem, but it is priced for laboratories. Wearable trackers such as Tobii Pro Glasses are sold by quotation, with a published entry price near ₹ 9.6 lakh for the previous generation.
- Even a consumer screen-mounted tracker such as the Tobii Eye Tracker 5 costs about ₹ 48,000 — still an unrealistic purchase for someone who needs it purely in order to communicate.
- EYEQ runs entirely on a commodity webcam of roughly ₹ 1,900 — hardware most users already own — with no infrared hardware, no client build step, and all gaze computation kept on the user's own device.
- Large, gaze-friendly key clusters make targets easy to hit despite noisy gaze, but several letters share one region, so distinct words collapse onto the same cluster code. A quantum conditional selection rule resolves that ambiguity, improving the accuracy with which the intended word is committed.



Objectives

1

Gaze estimation on ordinary hardware

To estimate gaze from MediaPipe iris and head-pose landmarks through a per-user ridge-regression calibration, using a commodity webcam with no infrared hardware.

2

Reliable, fatigue-aware selection

To commit characters by dwell selection over large, gaze-friendly key clusters that remain easy to hit under realistic gaze noise.

3

Higher accuracy on the intended word

To apply a quantum conditional selection rule that fuses gaze evidence with linguistic context, improving the accuracy with which the intended word is committed.

4

A guarantee, not a hope

To ensure through a consistency-preserving oracle that every decoded output re-encodes to exactly the clusters gazed at — never an off-target word — while staying within the webcam frame budget.

Abstract

Text entry still assumes working hands and a physical keyboard, excluding users with motor impairments and failing in hands-occupied or sterile settings. This project presents a eye-gaze typing system that runs entirely on a commodity webcam — no infrared hardware and no client build step. Gaze is estimated by mapping MediaPipe iris and head-pose landmarks through a per-user ridge-regression calibration, and characters are committed by dwell selection over large, gaze-friendly key clusters. Clustering makes targets easy to hit but renders the input ambiguous: several letters share one region, so distinct words collapse onto the same cluster code. To resolve this ambiguity accurately and in real time, we introduce a quantum conditional selection rule. Candidate targets form a superposition whose amplitudes carry the gaze likelihood; a language-conditioned controlled-unitary then fuses linguistic context through both amplitude and phase, and a Born-rule measurement commits the most probable candidate. Unlike classical Bayesian fusion, which assumes gaze–intention independence, the phase term admits interference between context-coherent hypotheses, while amplitude amplification accumulates per-frame evidence to shorten dwell time. A consistency-preserving oracle guarantees every decoded output re-encodes to exactly the gazed clusters, never an off-target word. The rule is realized as a lightweight classical state-vector simulation, keeping the full pipeline within the webcam frame budget.

KEYWORD:- Index Terms — Quantum conditional selection rule; quantum-inspired inference; interference-based gaze–language fusion; amplitude amplification; cluster-constrained decoding; eye-gaze typing; webcam gaze estimation; dwell-based selection; assistive text entry.

Proposed Methodology

1

Stage 1 - Gaze Sensing

MediaPipe Face Landmarker extracts 478 landmarks per webcam frame (GPU, falling back to CPU/WASM). Iris centres (landmarks 468, 473) plus eye-corner/lid points reduce to an 8-D feature vector — four iris-position ratios and head-pose (nose position, inter-eye width) — that stays stable as the user moves.

2

Stage 2 - Calibration and Mapping

A per-user ridge regression (Tikhonov-regularized, solved by Gauss-Jordan elimination with adaptive λ) maps the feature vector to screen coordinates. Calibrated from 7 points × 5 clicks, and continuously refined by ordinary clicks during use.

3

Stage 3 - Selection

Exponential smoothing plus a dwell-timer state machine commit a key once gaze rests on it. Keys are grouped into large, gaze-friendly clusters (9-cluster QWERTY grouping) so noisy gaze can still hit a target — but this collapses distinct letters, and distinct words, onto the same cluster code.

Proposed Methodology

4

Stage 4 - Gaze Sensing

After gaze clustering, multiple candidate words may correspond to the same cluster sequence. The candidate words are represented as quantum states, with their amplitudes initialized according to **gaze likelihood**. A **Quantum Bayesian Network** incorporates linguistic context, while a **Quantum Kernel/ VQC** evaluates the relationship between the current gaze features and candidate targets. The resulting amplitude and phase information is used to represent the combined gaze–language evidence, and the highest-probability candidate is selected through measurement. **QAOA** can subsequently optimize the candidate/typing sequence under the defined constraints.

5

Stage 5 - Consistency-Preserving Selection Guarantee

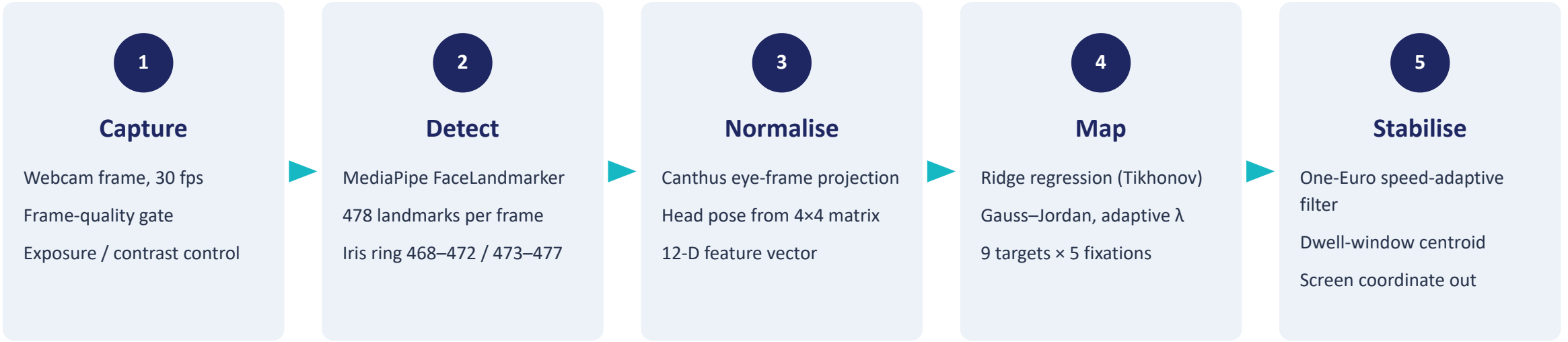
Before committing the selected word, a consistency constraint verifies that the candidate can reproduce the exact cluster sequence observed from the user's gaze.

Hardware and Software Requirements & Gantt Chart

Category	Component/ Requirement	Specification/ Version	Purpose/ Role	Cost
Hardware	Laptop/ computer	Intel/ AMD/ Apple Silicon, 8GB RAM	Model execution & processing	30,000 to 80,000
Hardware	Webcam	720p/ 1080p, 30FPS	Real-time eye tracking	1,900
Software	Python	3.10/ 3.11	Core development	Open source
Software	MediaPipe Face Mesh	0.10.21	Facial & iris landmark detection	Open source
Software	OpenCv	4.0.0 or above	Image processing & camera handling	Open source
Software	PyAutoGUI	Latest compatible version	Keyboard interaction	Open source

System Design

Block diagram 1 of 2 — gaze estimation pipeline, webcam frame to stabilised screen coordinate



FEATURE VECTOR	CALIBRATION	HELD-OUT GAZE ERROR	HARDWARE
12 dimensions	**9 targets × 5 fixations**	**4.37° of visual angle**	**Commodity webcam, no IR**

System Design

Block diagram 2 of 2 — from screen coordinate to a committed character

Key posterior

Exact 2-D Gaussian integral
over every key rectangle

Evidence fusion

P(key | gaze) × P(letter | prefix)^0.70
from a prefix trie and 3-gram

Reliability tempering

Quantum Bayesian Network grades
the frame; posterior is damped

Commit gate

Above θ = 0.80 the key is typed,
below it the system asks

Committed

Character enters the text buffer

Deferred

Three choice chips are offered instead of a guess

At the word boundary

Cluster code

Gazed regions form a digit string, e.g. hello → 51663

Hybrid decoder

Exact lookup, then lattice beam search scored by a bigram model

Consistency oracle

Accepts a word only if it re-encodes to exactly the gazed clusters

Module-wise Implementation Progress

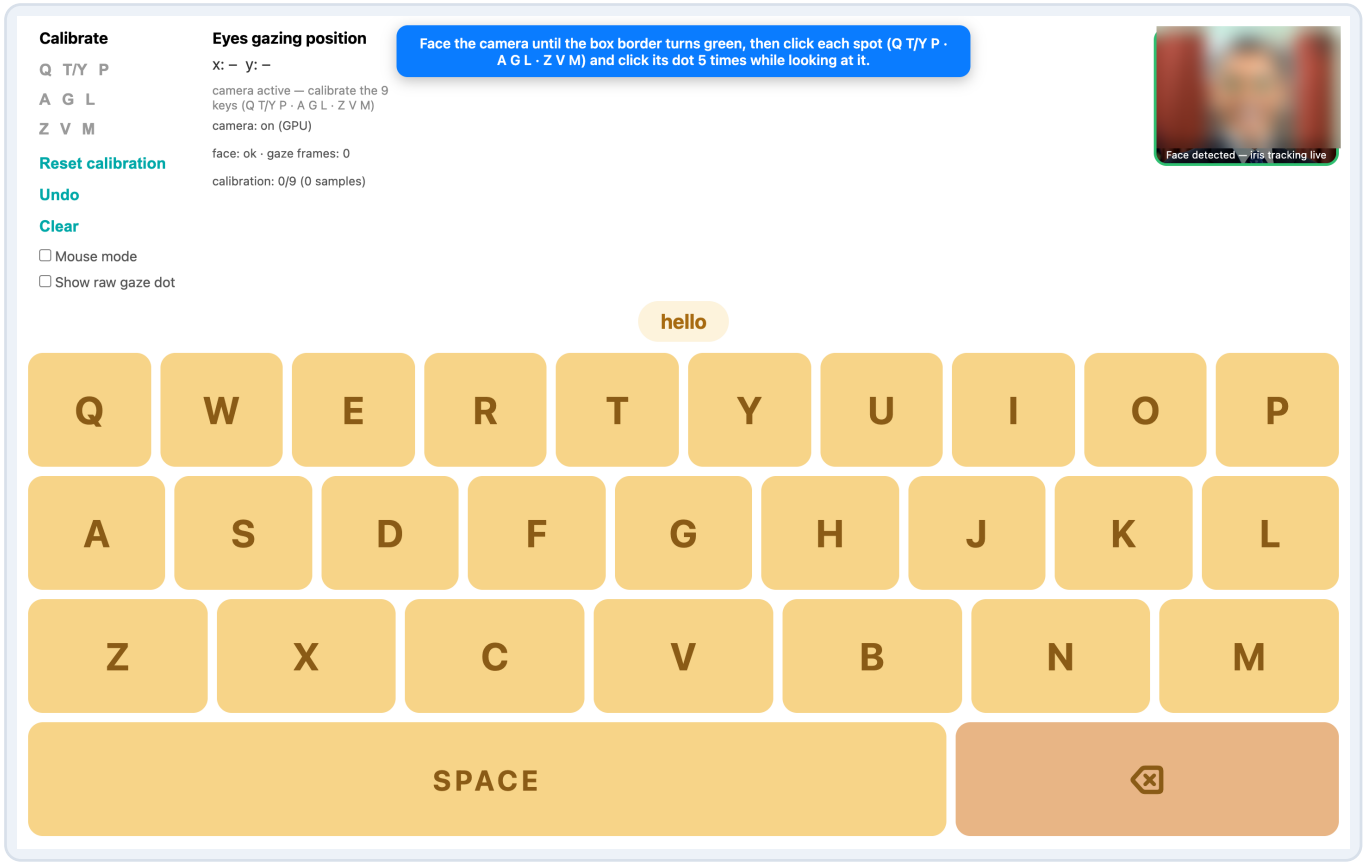
Each module, where it lives, and its state as of Review 2

Module	Implementation	Status
Gaze sensing and landmarks	MediaPipe FaceLandmarker, 478 points, iris ring · static/iris-tracker.js	**Complete**
Calibration and regression	12-D canthus features, ridge regression, coverage-aware sample eviction	**Complete**
Temporal filtering and dwell	One-Euro filter, dwell centroid, exit hysteresis · static/app.js	**Complete**
Key posterior and fusion	Exact Gaussian key integral fused with a prefix-trie letter prior	**Complete**
Cluster decoder and oracle	Three-tier decoder with re-encoding guarantee · clusters.py, decoder.py	**Complete**
Quantum modules	VQC, QSVM, QAOA, QBN, QNN, quantum kernel, conditional selection rule	**Implemented, measured**
Evaluation harness	Recorded-session benches, grouped splits, negative-result log · eval/	**Complete**
Web application	FastAPI service, nine endpoints, browser client, no build step	**Complete**

All modules are integrated in a single running service; the quantum modules are implemented and benchmarked against classical baselines rather than assumed.

Prototype / Working Demonstration

Captured from the running system — live camera, iris tracking and a committed word



1 Camera and tracking

Camera reports on (GPU); face detected and iris tracking live at 30 fps.

2 Calibration

Nine on-screen targets anchored to key centres (Q T/Y P · A G L · Z V M).

3 Dwell selection

A fill ring shows dwell progress; the key commits when the ring completes.

4 Committed text

The word hello was entered through the dwell and commit path, not by keyboard.

5 Word suggestion

A vocabulary chip is offered from the prefix trie as the word forms.

Testing Progress & Challenges Faced

Part 1 of 2 — what has been tested, and the measured result

5.97°	4.37°	+26.5%	21
Baseline gaze error	After canthus normalisation	Improvement, all sessions	Held-out targets per session

Test suite	What it verifies	Result
calibration_flow	Camera opens, face tracked, nine targets registered, model fitted	**All pass**
dwell_typing	Dwelling on a key types that key, in the correct order	**All pass**
gate_regression	The reliability gate refuses unusable frames and never locks out typing	**All pass**
qaoa_e2e	Word-level re-decoding returns a valid word within the latency budget	**All pass**
video_adjust	Exposure and contrast controls reach the tracker	**All pass**
gaze_bench + property tests	Recorded-session accuracy and 33 selection-rule properties	**All pass**

Accuracy is measured by leave-one-target-out over 21 positions on three recorded human sessions; splits are grouped by target so repeated fixations cannot leak between train and test.

Testing Progress & Challenges Faced

Part 2 of 2 — problems found by measurement, and how each was resolved

1 Vertical channel was contaminated

Normalising iris offset by the eyelid aperture couples gaze to a structure that moves on every blink.

Resolved Normalised against the canthi instead — error fell 5.97° to 4.37°, winning every session.

2 Calibration coverage was silently lost

Evicting the oldest sample shrank spatial coverage from thirteen grid cells to two.

Resolved Coverage-aware eviction with protected anchors; error recovered from 8.25° back to 4.87°.

3 Quantum kernels lost to classical ones

Fidelity between distinct samples falls from 0.128 to 0.014 as qubits rise from 4 to 12.

Resolved Exponential concentration identified; the classical RBF kernel was retained.

4 A classifier ran out of distribution

The movement gate was trained on synthetic kinematics; 56–66% of recorded values fall outside that range.

Resolved Diagnosed against recorded sessions; retraining on real kinematics is the next step.

5 A key is smaller than the gaze error

Measured error is 1.0–1.7× the width of one QWERTY key, so the commit gate defers rather than guessing.

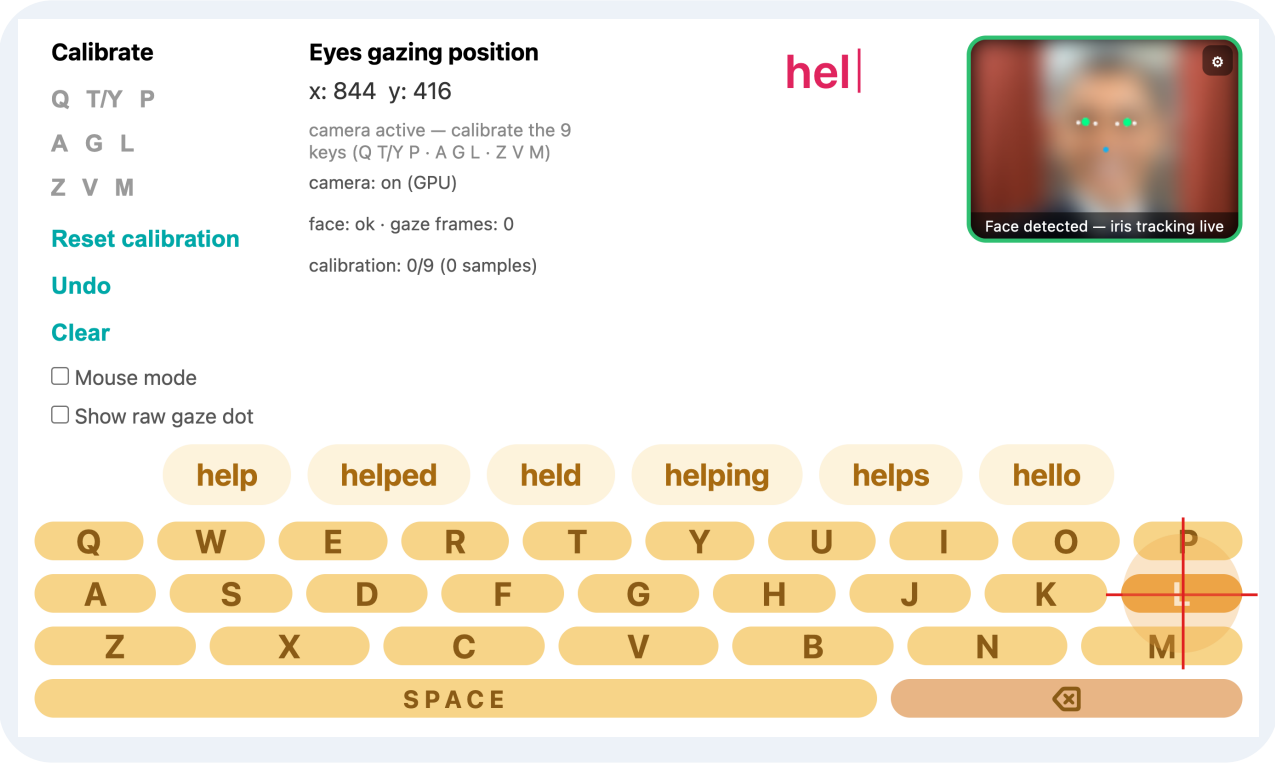
Resolved Choice chips carry the selection; larger cluster targets are the designed remedy.

Takeaway

Every design change in this phase was accepted or rejected on measured evidence from recorded human sessions, and each rejected idea is recorded so it is not retried on intuition.

Complete Project Implementation

Part 1 of 2 — the finished system, captured live, and how it is layered



Final interface: the tracking preview (top right) shows iris centres and canthi; the crosshair rests on L, filling its dwell ring; word suggestions sit above the keyboard. Face blurred for privacy.

1 Browser client `static/`

Face and iris landmarks, canthus features, calibration and ridge fit, One-Euro filter, dwell and commit. Runs at 30 fps in the browser; nothing to install.

2 Local server `app.py (FastAPI)`

Serves the keyboard; word completion from a 16,839-word vocabulary; cluster decoder with a consistency oracle; evaluation logging.

3 Research and evaluation `quantum/, eval/`

Quantum selection rule and the new quantum residual correction stage, each benchmarked against a classical baseline on the recorded sessions.

Testing & Final Results

Part 2 of 2 — accuracy on recorded human sessions, the measured outcome

1 Recorded human sessions

11 sessions from 9 participants (P01–P09), de-identified: landmarks and screen geometry only.

2 21 targets per session

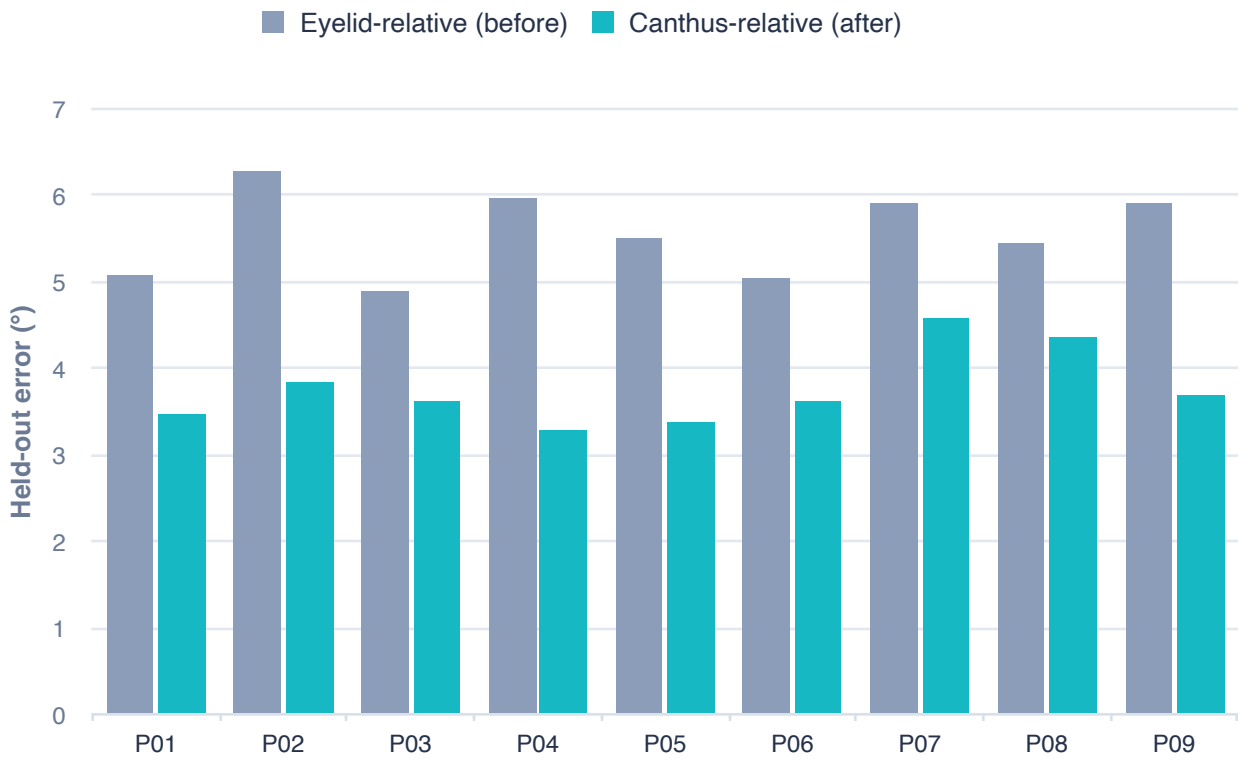
13 for fitting, 8 held out across corners, edges and centre; five 300 ms fixations each.

3 Grouped leave-one-target-out

A target's fixations are never split between training and test, so nothing leaks.

4 Error in degrees

Screen size and viewing distance are measured, so degrees of visual angle are exact.

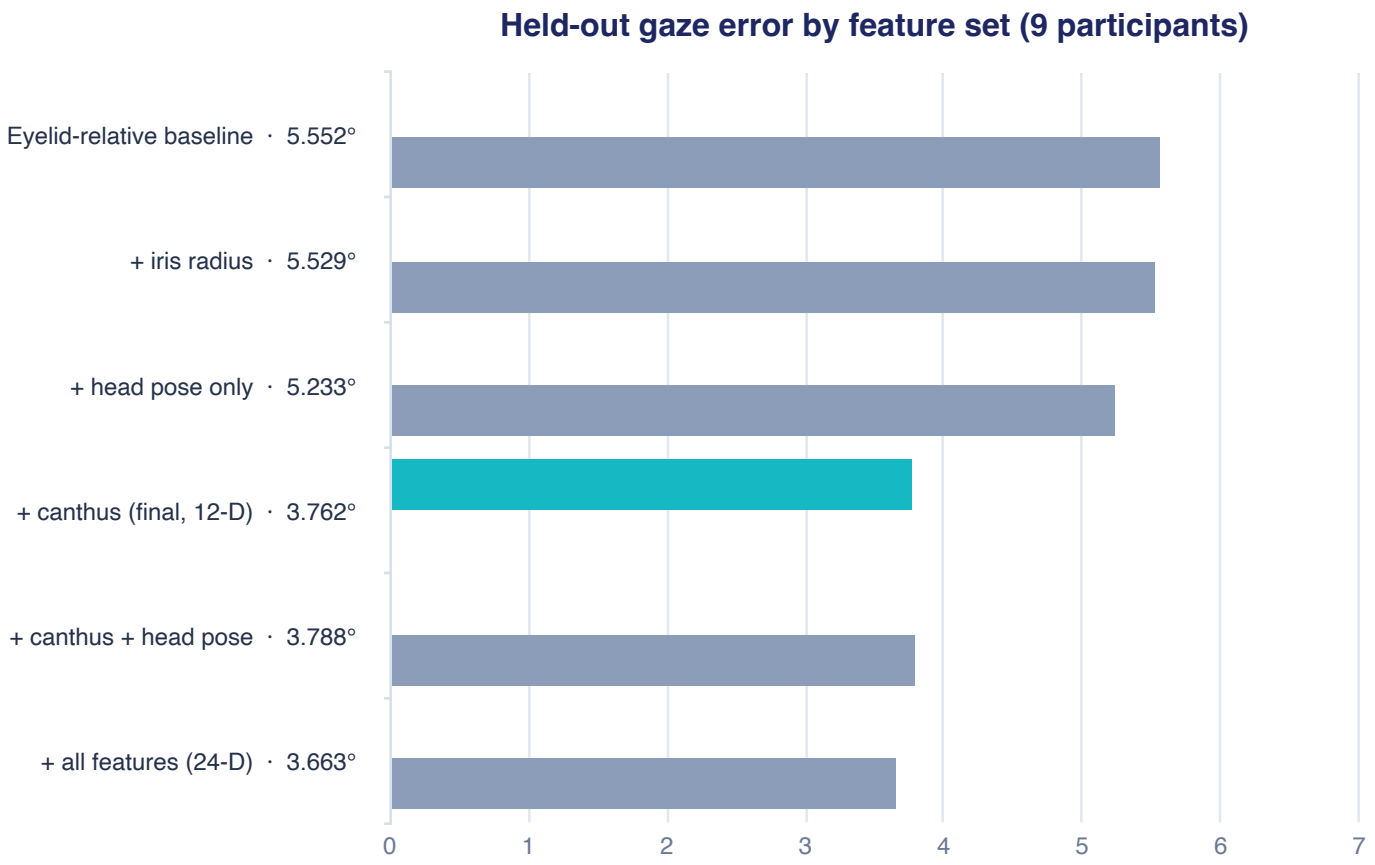


5.55° → 3.76° **–32.3 %, and every participant improved (19.5–44.6 %)**

Paired over participants: $t(8) = 9.31$, $p = 1.4 \times 10^{-5}$; Wilcoxon $p = 0.004$

Final Results & Performance Analysis

Part 1 of 3 — gaze accuracy: which feature caused the gain



1 The canthi are the cause

Iris radius alone gives +0.4 %, head pose alone +5.7 %. The canthus frame gives +32.3 %, and removing it costs 40–47 %.

2 Head pose becomes unnecessary

Added on top of the canthus frame, pose changes nothing detectable ($p = 0.44$): the frame already moves with the head.

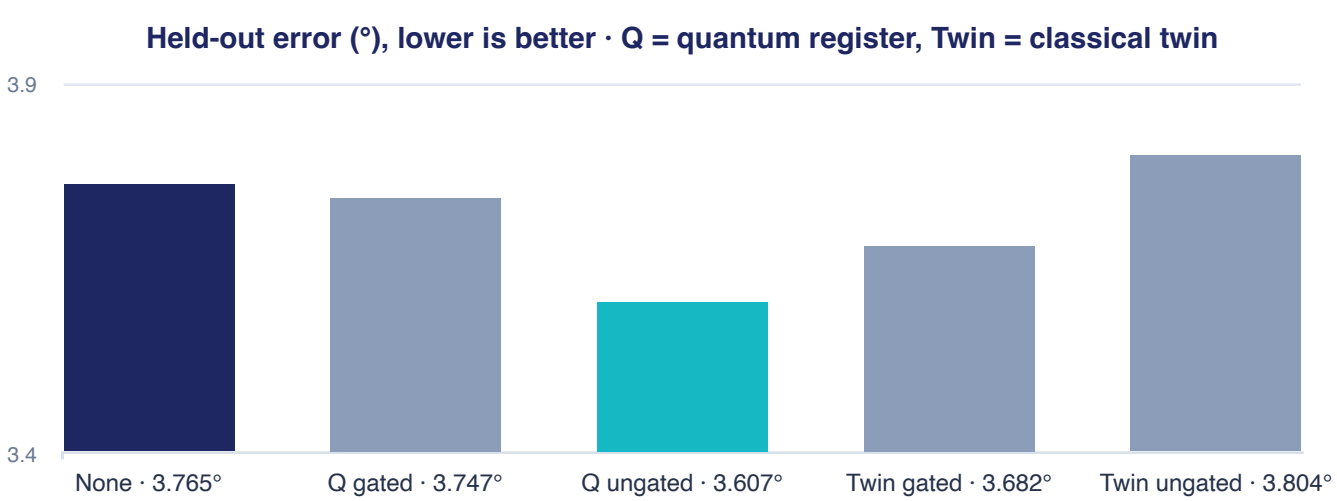
3 Roll-invariant by construction

Rotating a face through 0–40° moves the canthus features by at most 2×10^{-15}; an image-axis version moves by 0.069.

Final Results & Performance Analysis

Part 2 of 3 — every quantum component, measured against a classical baseline

Quantum component	Compared against	Measured result	Verdict
Conditional selection rule	Log-linear fusion	Median –0.03 percentage points over 20 conditions; 0 wins, 0 losses	**No gain**
Amplitude amplification	No amplification	Dwell 2.63 → 2.80 frames; accuracy 98.4 → 96.4 %	**Not deployed**
Quantum fidelity kernels	Linear ridge	131–254 % worse at 4–10 qubits; similarity 0.128 → 0.014 (4 → 12 qubits)	**Rejected**
Quantum residual register (new)	No correction	3.76° → 3.61° (–4.2 %), all 9 participants improved, p = 0.0004	**Promising**

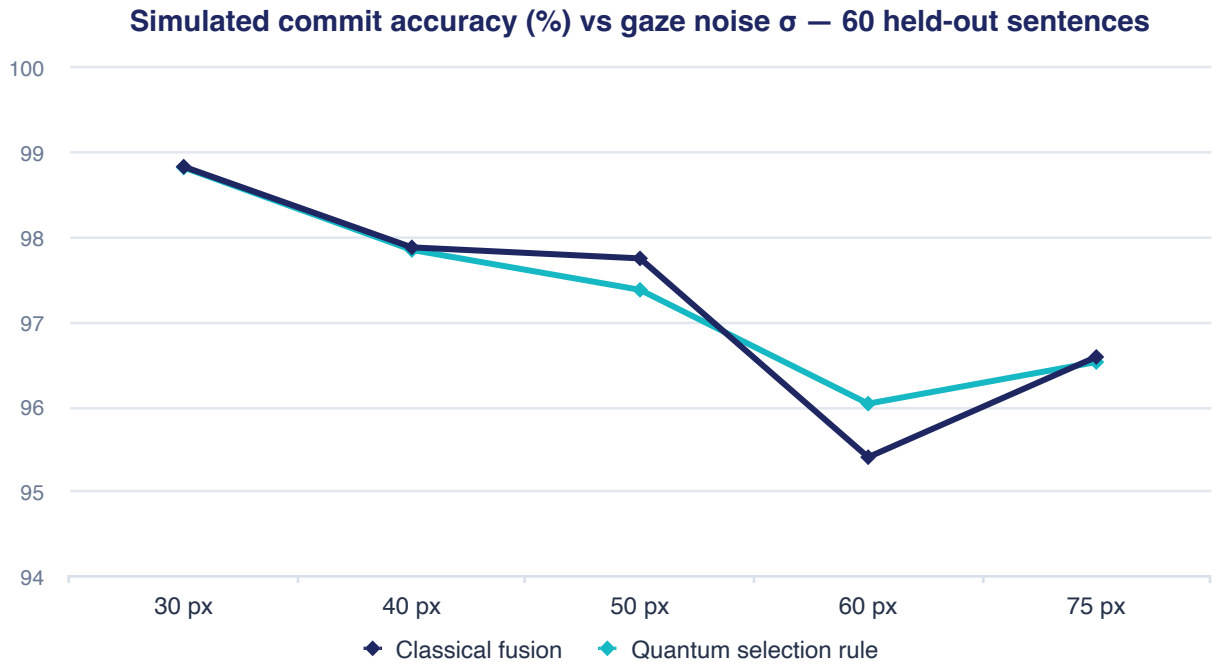


The one quantum stage that helped

The residual register lowered error for all nine participants. Its gain over an equal-size classical model (109 vs 102 parameters) is 5.2 % but not yet significant (p = 0.071), so it stays out of the typing path until it is confirmed on new recordings.

Final Results & Performance Analysis

Part 3 of 3 — typing performance: simulation, live use, and cost



Live typing — one participant, two short trials

Trial 1 11 commits · 18.2 % correct · 50 % prompts · 0.83 WPM

Trial 2 3 commits · 33.3 % correct · 40 % prompts · 0.56 WPM

Live accuracy is far below the 95–99 % simulated, so the gap had to be explained before it could be fixed.

Diagnosis: posture drift, not the model

With a 28-key calibration the error was 2.07° within one sitting. After the user shifted posture it rose above 8°, and 7.6° of that was one shared offset. A five-dot re-centre cut it from 8.74° to 4.70° and raised key hits from 4 % to 26 % (one session, experimental branch).

Next: validate the re-centre on more participants before merging it.

~0.2 ms	~0.02 ms	33 ms
Quantum rule, per keystroke	Classical fusion, per keystroke	Frame budget at 30 fps

Thank You